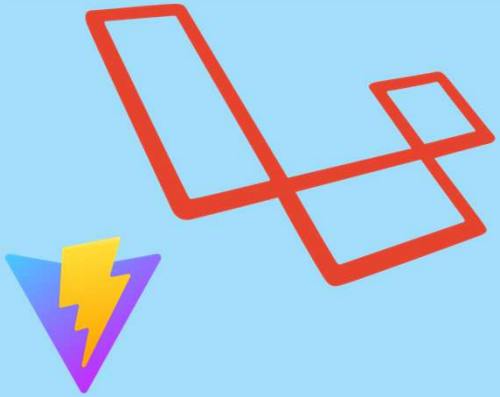


Laravel

Tecnologías Avanzadas de Desarrollo

UD3: Frameworks – Laravel: Sistema de Autenticación, Paneles de Administración y SMTP

Olga M. Moreno Martín

Licencia

Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

https://creativecommons.org/licenses/by-nc-nd/4.0/

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

Laravel

1. Introducción
2. Fortify – Sistema de autenticación
    1. Login/Register
3. MailTramp
    1. Forgot/Reset Password
4. Administración – Paneles
    1. Crítica a los existentes
    2. Puntos comunes

Introducción

Partiendo de la premisa de que ya sabemos crear un proyecto haciendo uso del framework Laravel en su versión 9, vamos a crear un sistema de autenticación partiendo de un proyecto base. Nos será muy útil en el futuro, dado que contendrá los básicos del sistema, esto es algo común a todos los proyectos web, y cada vez que trabajemos con un framework debemos implementar estas funcionalidades.

Para ayudarnos usaremos Fortify, un paquete de Laravel que proporciona una fácil integración de la autenticación y el registro de usuarios. Fortify incluye un conjunto de controladores y rutas predefinidas para manejar la autenticación y el registro de usuarios de manera rápida y sencilla.

Y además, será necesaria la gestión de la recuperación de la contraseña y debemos crear una conexión con un servidor SMTP, para ello usaremos MailTramp.

Por último, nos centraremos en las necesidades que debe cubrir un panel de administración, criticando los diseños UX/UI establecidos y sacando puntos comunes para establecer un patrón de desarrollo de interfaces lo más óptimo posible.

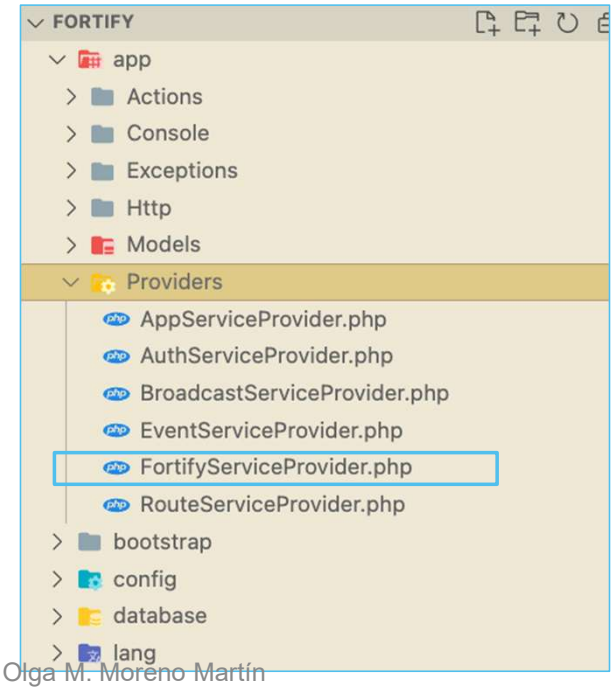
Fortify

Para instalar Fortify con Composer desde la terminal de comandos ejecuta la siguiente instrucción:

composer require laravel/fortify

Luego para que se publiquen los assets de Fortify:

php artisan vendor:publish --provider="Laravel\Fortify\FortifyServiceProvider"



Fortify

Métodos de FortifyServiceProvider:

```
public function boot(): void
    {
        Fortify::createUsersUsing(CreateNewUser::class);
        Fortify::updateUserProfileInformationUsing(UpdateUserProfileInformation::class);
        Fortify::updateUserPasswordsUsing(UpdateUserPassword::class);
        Fortify::resetUserPasswordsUsing(ResetUserPassword::class);

        RateLimiter::for('login', function (Request $request) {
            $email = (string) $request->email;

            return Limit::perMinute(5)->by($email.$request->ip());
        });

        RateLimiter::for('two-factor', function (Request $request) {
            return Limit::perMinute(5)->by($request->session()->get('login.id'));
        });
    }
```

Fortify

Abre el archivo config/app.php y busca la sección Application Service Providers y agregaremos la siguiente línea de código, para agregar Fortify al resto de Providers de nuestra app:

App\Providers\FortifyServiceProvider::class,

```
/*
 * Application Service Providers...
 */
App\Providers\AppServiceProvider::class,
App\Providers\AuthServiceProvider::class,
// App\Providers\BroadcastServiceProvider::class,
App\Providers\EventServiceProvider::class,
App\Providers\RouteServiceProvider::class,
App\Providers\FortifyServiceProvider::class,
```

Fortify

Si ejecutamos php artisan route:list, podemos ver un montón de rutas disponibles:

htdocs — bash — 172×37

```
                                                    Showing [29] routes

sh-3.2# php artisan route:list

  GET|HEAD  / ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... .....
  POST      _ignition/execute-solution ..... ignition.executeSolution › Spatie\LaravelIgnition › ExecuteSolutionController
  GET|HEAD  _ignition/health-check ..... ignition.healthCheck › Spatie\LaravelIgnition › HealthCheckController
  POST      _ignition/update-config ..... ignition.updateConfig › Spatie\LaravelIgnition › UpdateConfigController
  GET|HEAD  api/user ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... ..... .....
  GET|HEAD  forgot-password ..... password.request › Laravel\Fortify › PasswordResetLinkController@create
  POST      forgot-password ..... password.email › Laravel\Fortify › PasswordResetLinkController@store
  GET|HEAD  login ..... login › Laravel\Fortify › AuthenticatedSessionController@create
  POST      login ..... Laravel\Fortify › AuthenticatedSessionController@store
  POST      logout ..... logout › Laravel\Fortify › AuthenticatedSessionController@destroy
  GET|HEAD  register ..... register › Laravel\Fortify › RegisteredUserController@create
  POST      register ..... Laravel\Fortify › RegisteredUserController@store
  POST      reset-password ..... password.update › Laravel\Fortify › NewPasswordController@store
  GET|HEAD  reset-password/{token} ..... password.reset › Laravel\Fortify › NewPasswordController@create
  GET|HEAD  sanctum/csrf-cookie ..... sanctum.csrf-cookie › Laravel\Sanctum › CsrfCookieController@show
  GET|HEAD  two-factor-challenge ..... two-factor.login › Laravel\Fortify › TwoFactorAuthenticatedSessionController@create
  POST      two-factor-challenge ..... Laravel\Fortify › TwoFactorAuthenticatedSessionController@store
  GET|HEAD  user/confirm-password ..... Laravel\Fortify › ConfirmablePasswordController@show
  POST      user/confirm-password ..... password.confirm › Laravel\Fortify › ConfirmablePasswordController@store
  GET|HEAD  user/confirmed-password-status ..... password.confirmation › Laravel\Fortify › ConfirmedPasswordStatusController@show
  POST      user/confirmed-two-factor-authentication ..... two-factor.confirm › Laravel\Fortify › ConfirmedTwoFactorAuthenticationController@store
  PUT       user/password ..... user-password.update › Laravel\Fortify › PasswordController@update
  PUT       user/profile-information ..... user-profile-information.update › Laravel\Fortify › ProfileInformationController@update
  POST      user/two-factor-authentication ..... two-factor.enable › Laravel\Fortify › TwoFactorAuthenticationController@store
  DELETE    user/two-factor-authentication ..... two-factor.disable › Laravel\Fortify › TwoFactorAuthenticationController@destroy
  GET|HEAD  user/two-factor-qr-code ..... two-factor.qr-code › Laravel\Fortify › TwoFactorQrCodeController@show
  GET|HEAD  user/two-factor-recovery-codes ..... two-factor.recovery-codes › Laravel\Fortify › RecoveryCodeController@index
  POST      user/two-factor-recovery-codes ..... Laravel\Fortify › RecoveryCodeController@store
  GET|HEAD  user/two-factor-secret-key ..... two-factor.secret-key › Laravel\Fortify › TwoFactorSecretKeyController@show

                                                    Showing [29] routes

sh-3.2#
```

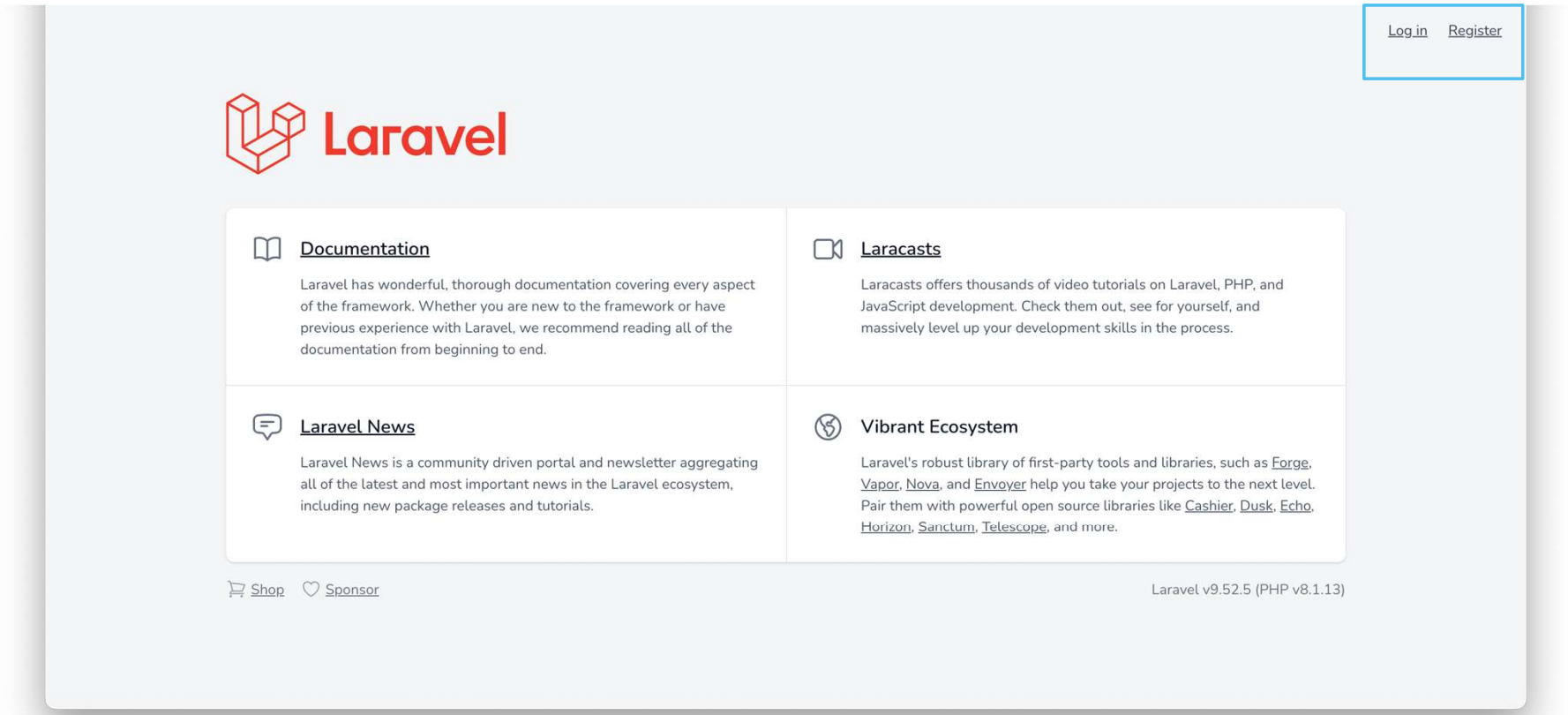
Fortify

Ahora podemos ejecutar las migraciones: php artisan migrate

```
htdocs — bash — 172×31
sh-3.2# clear

sh-3.2# php artisan migrate

  INFO  Preparing database.

  Creating migration table ..................................................... 42ms DONE

  INFO  Running migrations.

  2014_10_12_000000_create_users_table ......................................... 27ms DONE
  2014_10_12_100000_create_password_resets_table ............................... 32ms DONE
  2014_10_12_200000_add_two_factor_columns_to_users_table ...................... 20ms DONE
  2019_08_19_000000_create_failed_jobs_table ................................... 41ms DONE
  2019_12_14_000001_create_personal_access_tokens_table ........................ 47ms DONE

sh-3.2#
```

Fortify

Y luego levantar nuestro servidor con: php artisan serve



Fortify

Dentro de FortifyServiceProvider, en el método boot() debemos agregar:

```
Fortify::loginView(function(){
    return view('auth.login');
});
Fortify::registerView(function(){
    return view('auth.register');
});
```

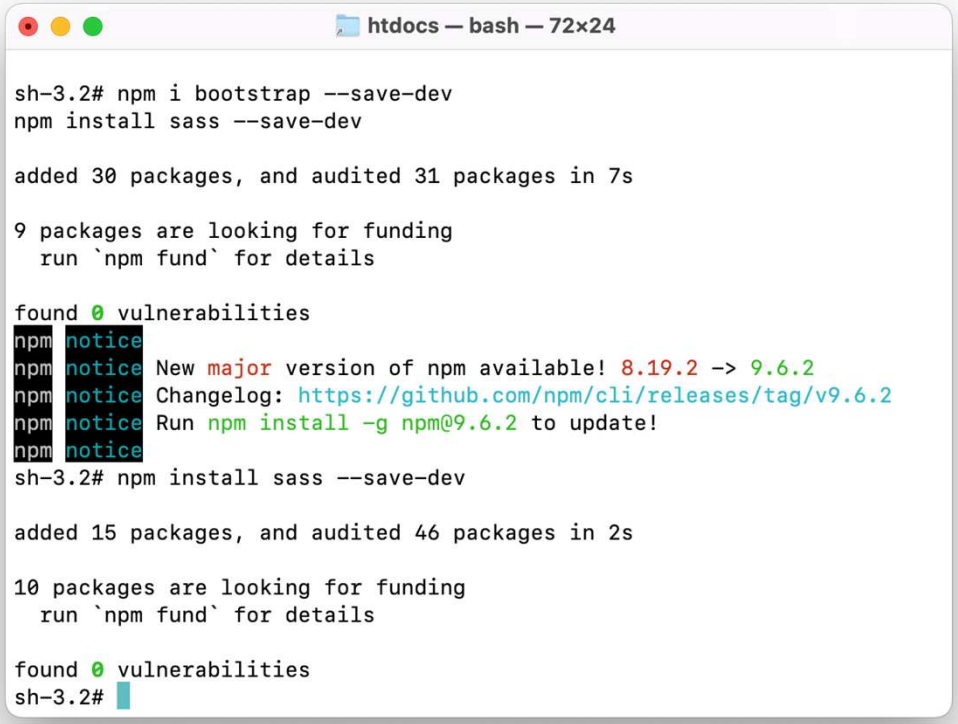
Y ahora vamos a crear las 2 vistas correspondientes.

Fortify

Primero vamos a instalar Bootstrap y Sass:

npm i bootstrap --save-dev

npm install sass --save-dev



Fortify

Recordatorio:

Desde tu editor de textos abre tu proyecto en Laravel 9 y cambia el nombre al archivo resources/css/app.css por resources/css/app.scss o crealo nuevo.

Y en ese mismo archivo app.scss importa Bootstrap 5 agregando la siguiente línea de código: @import 'bootstrap/scss/bootstrap';

Haz lo mismo en el archivo resources/js/app.js agregando en él:

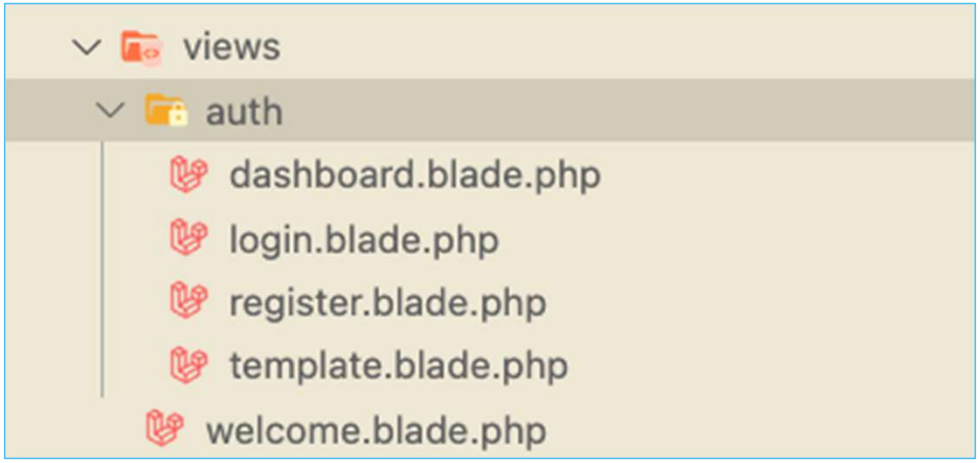
import * as bootstrap from 'bootstrap’

Ahora hay que configurar Vite, para ello abre el archivo vite.config.js que se encuentra en la raíz del proyecto y cambia las referencias del campo input:

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css',
                'resources/css/app.scss',
                'resources/js/app.js'],
            refresh: true,
        }),
    ],
});
```

Fortify

Una vez hecho esto creamos nuestras vistas:



Los ficheros están adjuntos a la práctica: auth.zip

Fortify

Abre el archivo routes/web.php y agrega la ruta para el dashboard el cual el middleware auth estará protegiendo la ruta para que solamente usuarios autenticados ingresen:

```
Route::get('/home', function () {
return view(‘auth.dashboard');
})->middleware('auth’);
```

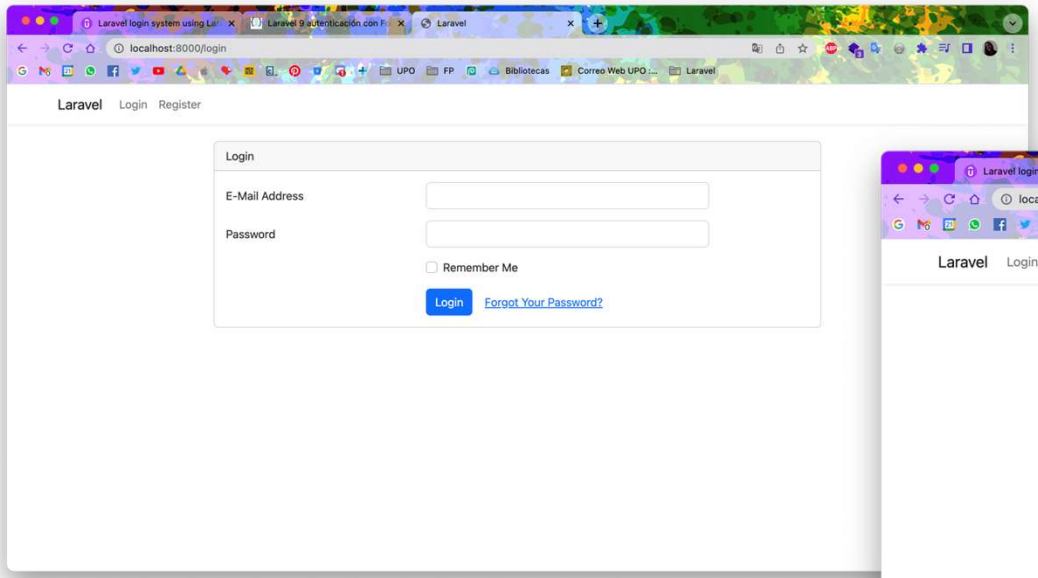
Para que nuestras vistas funcionen tiene que estar funcionando Node.js, dado que necesitamos npm para que Vite funcione:

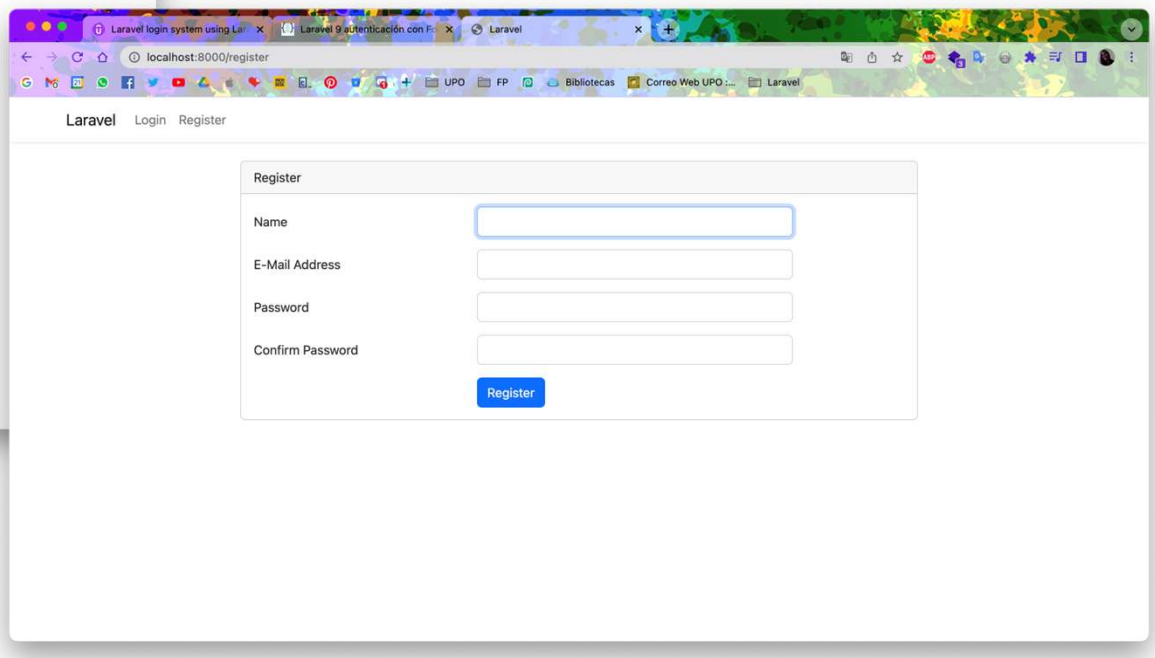
```
npm run dev
```

Así que tendremos corriendo tanto npm como artisan.

Fortify

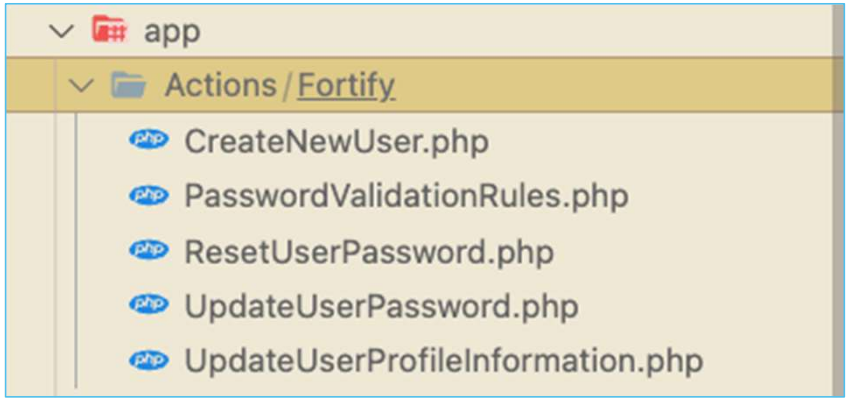
Si todo ha ido bien veremos algo así:





Fortify

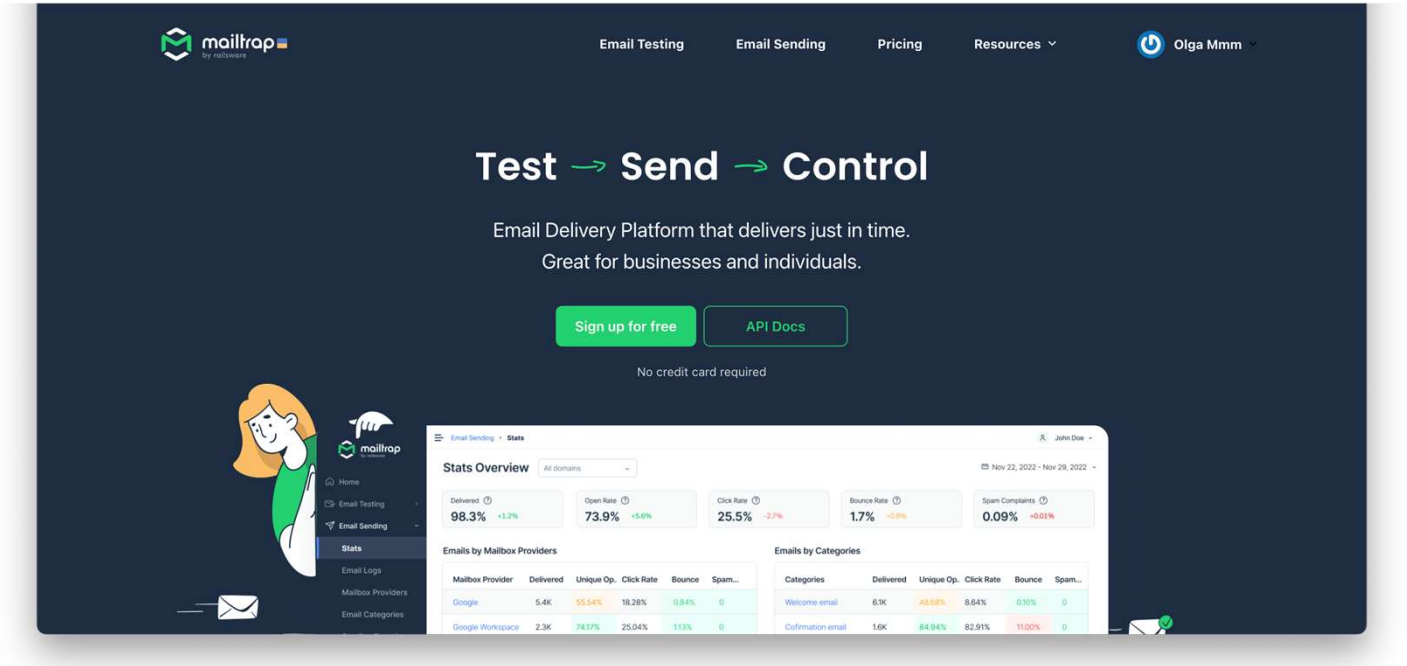
En App\Actions\Fortify podemos encontrar todos los archivos para validar los datos de los formularios.



Fortify

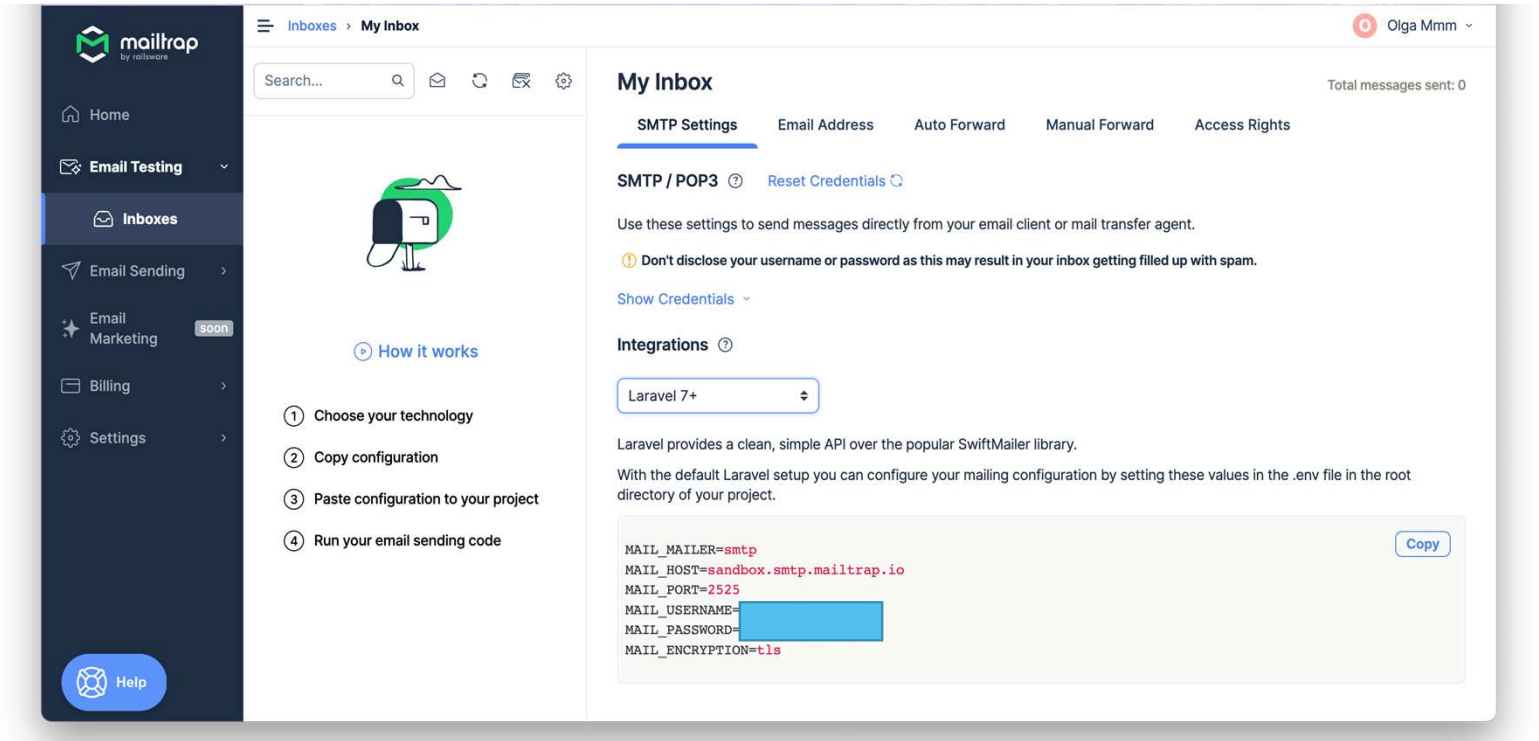
Ahora veremos como recuperar una contraseña olvidada, para ello, necesitamos configurar un servidor SMTP.

Vamos a usar MailTrap.



MailTrap

Una vez nos hemos registrado debemos crear una inbox nueva, seleccionamos integración para Laravel y copiamos los datos que nos da. Nos van a servir para configurar nuestro .env. Hay que sustituir los datos por los que nos da Mailtrap.

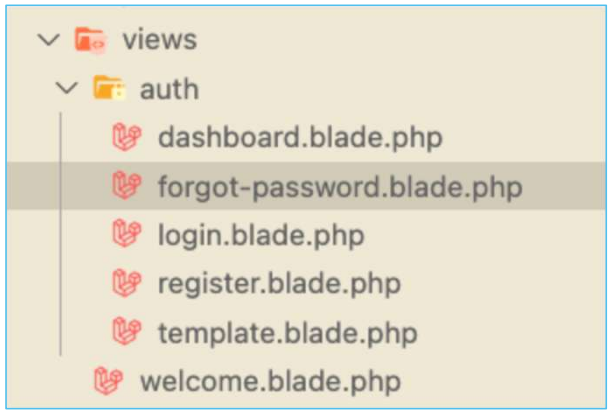


Fortify

Ahora debemos abrir FortifyServiceProvider, y agregar el siguiente código al método boot:

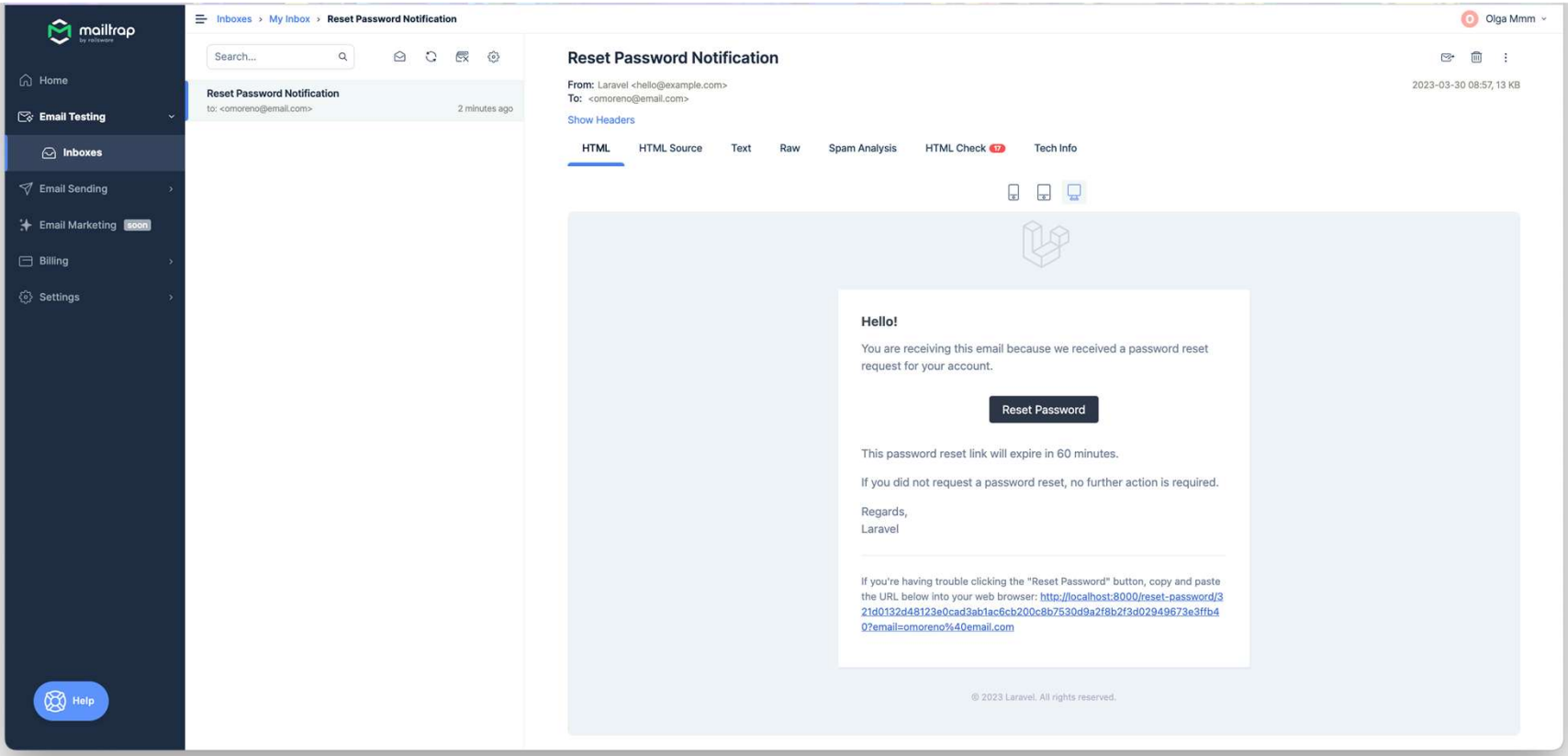
```
Fortify::requestPasswordResetLinkView(function(){
return view('auth.forgot-password');
});
```

Ahora, nos toca crear la vista correspondiente, que está en el archivo adjunto a la práctica.



MailTrap

Si todo ha ido como se esperaba veremos un nuevo mensaje en nuestra bandeja de entrada.



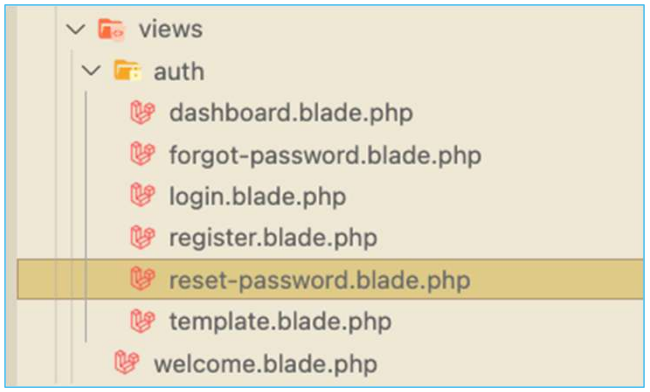
Fortify

Tras recibir el email necesitaremos otra vista, para poder resetear la contraseña.

Por tanto, debemos dirigirnos de nuevo a FortifyServiceProvider, y agregar el siguiente código al método boot:

```
Fortify::resetPasswordView(function($request){
return view('auth.reset-password',['request'=> $request]);
});
```

Ahora, nos toca crear la vista correspondiente, que está en el archivo adjunto a la práctica.



Fortify

En config/fortify.php encontramos features, debe quedar así:

```
'features' => [
    Features::registration(),
    Features::resetPasswords(),
    Features::emailVerification(),
    // Features::updateProfileInformation(),
    // Features::updatePasswords(),
    // Features::twoFactorAuthentication([
    //     'confirm' => true,
    //     'confirmPassword' => true,
    //     // 'window' => 0,
    // ]),
],
```

Fortify

Descomentar esta línea en User.php

```
use Illuminate\Contracts\Auth\MustVerifyEmail;
```

Y agregar:

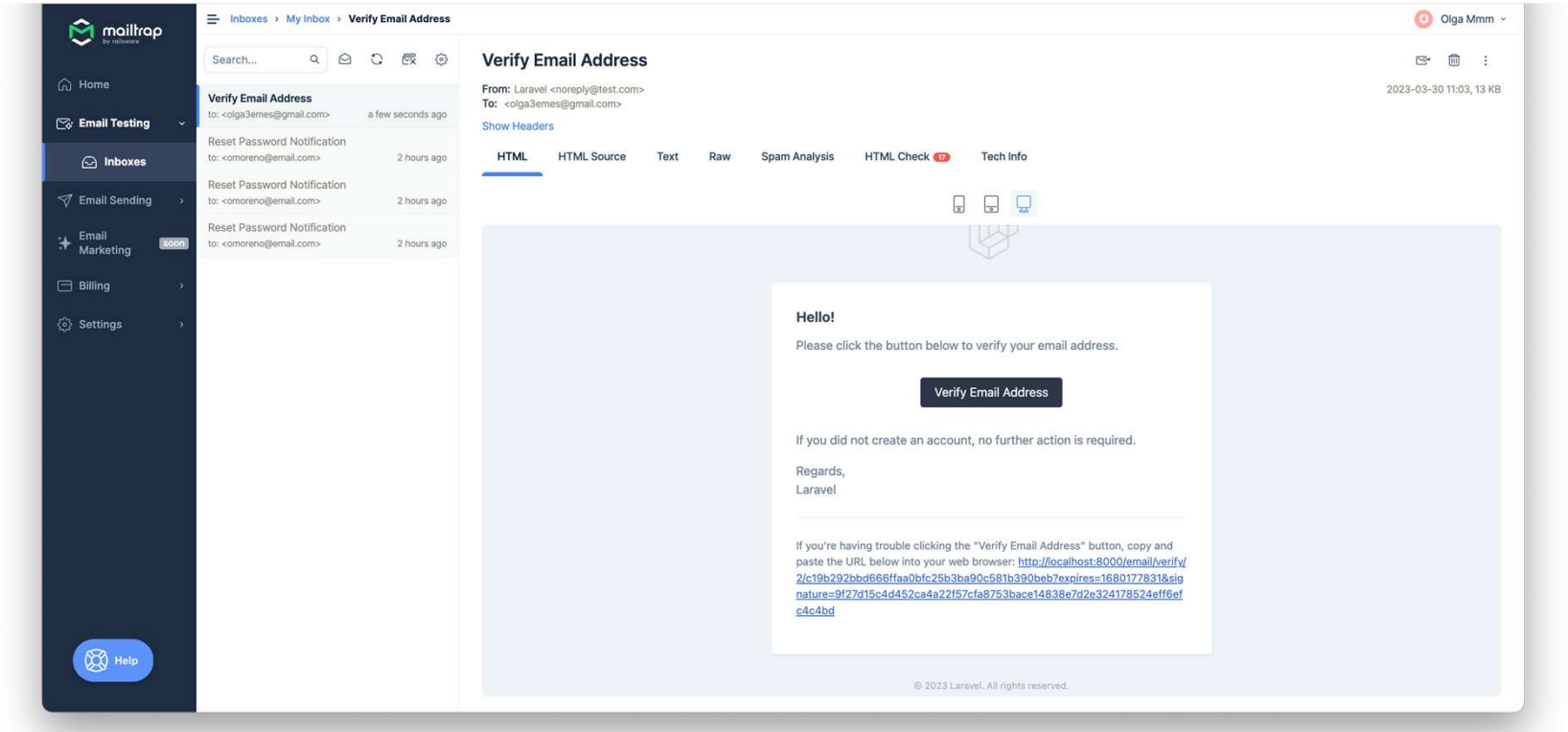
```
class User extends Authenticatable implements MustVerifyEmail
{
```

En web.php tenemos que pedir que el correo electrónico sea verificado:

```
Route::get('/home', function () {
    return view('auth.dashboard');
})->middleware(['auth','verified']);
```

MailTrap

Si nos registramos obtendremos:



Fortify

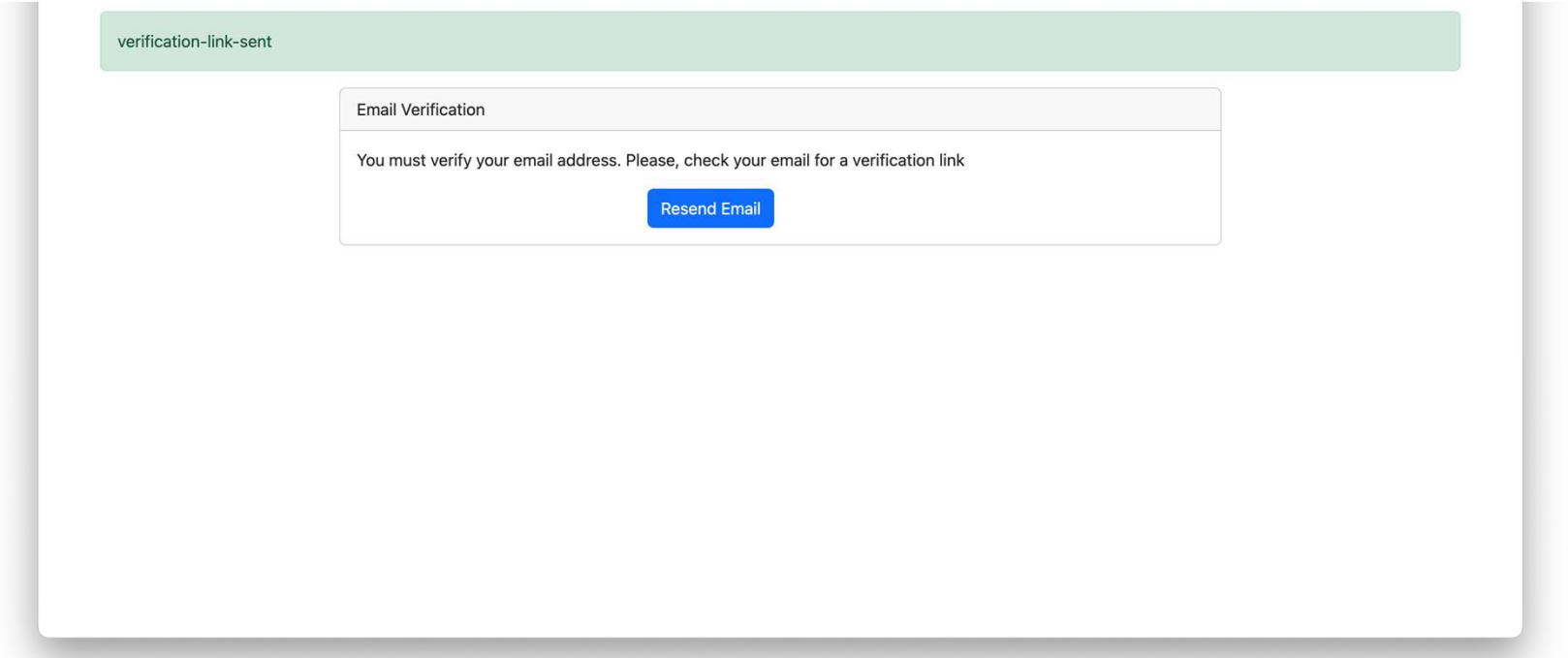
Ahora necesitamos una nueva vista:

```
Fortify::verifyEmailView(function(){
return view('auth.verify-email');
});
```

Encontrarás el código en el archivo adjunto a la práctica.

Fortify

Si todo ha sido correcto veremos esto:



Panel de administración

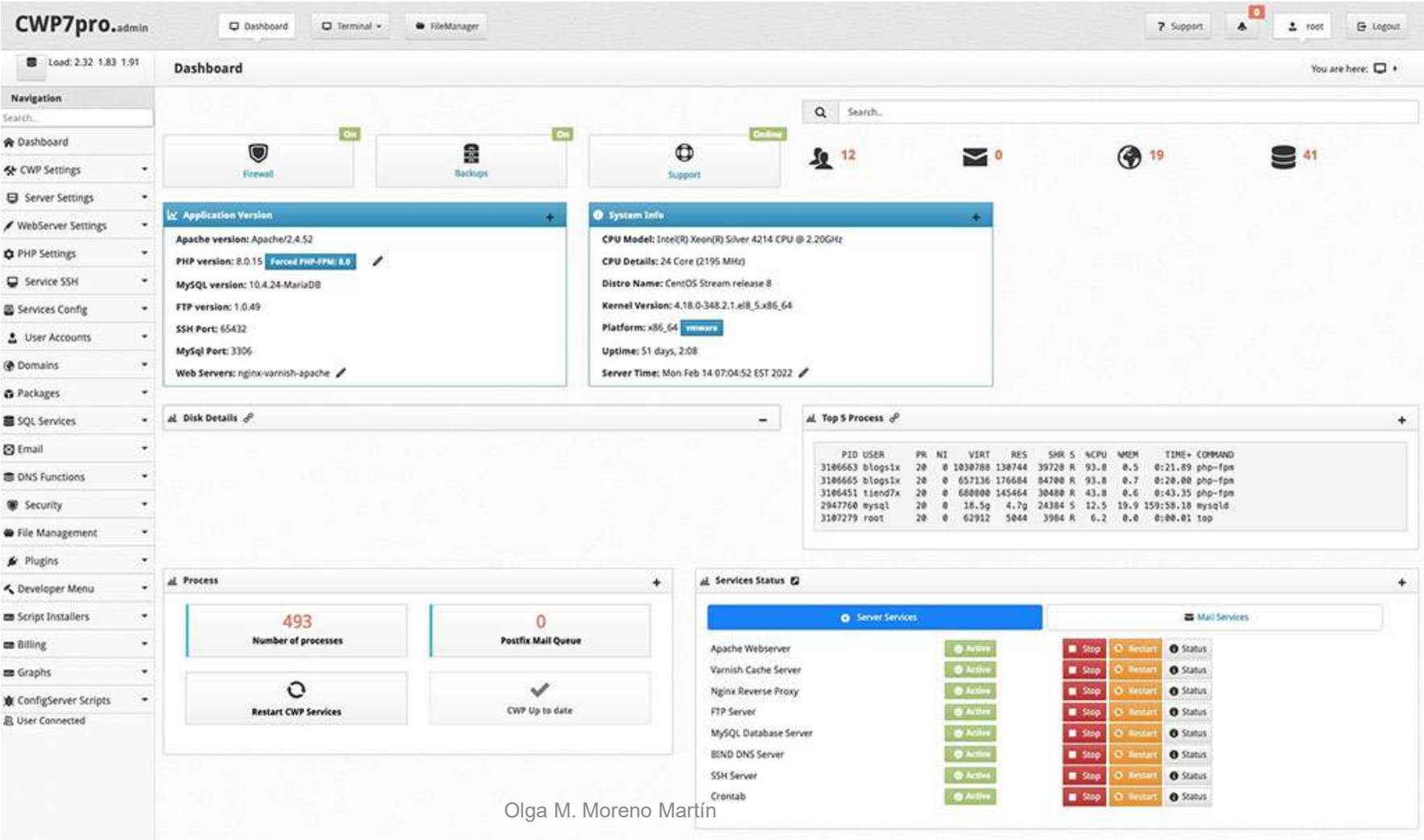
Como no diseñar un panel de administración (Experiencia Personal)

Más que como no diseñarlo la idea es evitar hacer algo así, un ejemplo (bajo todo mi respeto) podría ser el Joomla es un gestor de contenidos completísimo, de los mejores que hay, pero las veces que lo he usado he visto que si el usuario que ha de acceder al panel no sabe “de que va esto” y normalmente le cuesta mucho, pero mucho, encontrar las cosas y es más saber usar y entender cada opción que otra cosa.

Esto pasa en muchísimos sistemas, complejidades muy altas, que hacen que muchos usuarios nos digan “es que yo soy muy torpe y no me entiendo con la aplicación”.

¿Cuántas veces hemos escuchado algo así?

Ejemplo:


CWP7pro.admin
Dashboard
Terminal
FileManager
Support
root
Logout
Load: 2.32 1.83 1.91
Dashboard
You are here:
Navigation
Search...
Dashboard
CWP Settings
Server Settings
WebServer Settings
PHP Settings
Service SSH
Services Config
User Accounts
Domains
Packages
SQL Services
Email
DNS Functions
Security
File Management
Plugins
Developer Menu
Script Installers
Billing
Graphs
ConfigServer Scripts
User Connected
Firewall
Backups
Support
12
0
19
41
Application Version
Apache version: Apache/2.4.52
PHP version: 8.0.15
MySQL version: 10.4.24-MariaDB
FTP version: 1.0.49
SSH Port: 65432
MySql Port: 3306
Web Servers: nginx-varnish-apache
System Info
CPU Model: Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz
CPU Details: 24 Core (2195 MHz)
Distro Name: CentOS Stream release 8
Kernel Version: 4.18.0-348.2.1.el8_5.x86_64
Platform: x86_64
Uptime: 51 days, 2:08
Server Time: Mon Feb 14 07:04:52 EST 2022
Disk Details
Top 5 Process
Process
493
Number of processes
0
Postfix Mail Queue
Restart CWP Services
CWP Up to date
Services Status
Server Services
Mail Services
Apache Webserver
Varnish Cache Server
Nginx Reverse Proxy
FTP Server
MySQL Database Server
BIND DNS Server
SSH Server
Crontab

Panel de administración

¿Que causa esto?

En mi opinión lo que causa esto es el exceso de cosas, el usuario está sobreestimulado, colores, formas e iconos totalmente diferentes, la mirada se desvía, el usuario se distrae y no va directo a donde tiene que ir.

También otras cosas como la forma de explicar las opciones, el vocabulario utilizado, etc.

Con esto no quiero, ni mucho menos ofender a nadie ni desprestigiar el trabajo de otros desarrolladores, simplemente comentar que si lo debe usar un usuario que no entiende mucho de webs va a costarle y solo faltaría hacer una web y que esa persona/s no pueda/n usarla bien porque nos hemos pasado de complejidad en el diseño para ellos.

Panel de administración

La palabra SENCILLO tiene que ser la primordial.

Tan fáciles de entender que casi no haga falta explicar nada sobre él. Si lo conseguimos, entonces hemos hecho nuestro trabajo satisfactoriamente.

No olvidemos que los paneles tienen que cumplir con su funcionalidad, y que no podemos desviar el foco de ella.

¿Cómo lograrlo? EXPERIENCIA y ORDEN

Pero… ¿podemos tomar la experiencia de otros? ¡Pues claro que sí!

¿ser ordenados?… Pues tenemos que mejorar en ello, hay que organizarlo todo muy bien.

Paneles de administración

Ejemplos de plantillas existentes:

https://vetra.laborasyon.com/demos/default/index.html

http://rubick.left4code.com/

https://themesbrand.com/skote/layouts/ecommerce-products.html

http://68.183.209.47/materialadmin/?storefront=envato-elements#/dashboard

Crítica constructiva

Olga M. Moreno Martín